DAG Pipeline Declared by `pipeline.yaml`

DAG Pipeline Declared by `pipeline.yaml` I

Single-project pipelines read `infrastructure/core/pipeline/pipeline.yaml`. `scripts/runner/execute_pipeline.py` expands the declarative DAG, applies tag filters (`--core-only` skips `llm` stages), checkpoints between nodes, then dispatches the declared `scripts/pipeline/stage_NN_*.py` scripts or builtin methods (`_run_clean_outputs`).

The **default YAML graph contains ten named core stages** (plus telemetry configuration metadata):

DAG Pipeline Declared by `pipeline.yaml` I

1. **Clean Output Directories** — wipes prior `projects/<name>/output/` + delivered `output/<name>/` paths so stale PDFs cannot satisfy validation.
2. **Environment Setup** (`scripts/pipeline/stage_00_setup.py`) — Python/uv probing, toolchain discovery, scaffolding directories, `PYTHONPATH` wiring.
3. **Infrastructure Tests** (`scripts/pipeline/stage_01_test.py --infra-only`) — `tests/` suite with the configured $\geq$60% source-coverage floor.
4. **Project Tests** (`scripts/pipeline/stage_01_test.py --project-only`) — isolated per-project suites with each project's declared floor ($\geq$90% for this exemplar).
5. **Project Analysis** (`scripts/pipeline/stage_02_analysis.py`) — lexicographically ordered `projects/<name>/scripts/*.py`, each a thin orchestrator (`src/` does real work).
6. **PDF Rendering** (`scripts/pipeline/stage_03_render.py`) — Pandoc $\rightarrow$ XeLaTeX loop, bibliography assembly, injected variables from Stage 02 artefacts.
7. **Output Validation** (`scripts/pipeline/stage_04_validate.py`) — PDF structure, manifests, Markdown hygiene.
8. **LLM Scientific Review** (`scripts/pipeline/stage_06_llm_review.py --reviews-only`; `tags: llm`) — executive + quality critiques via local Ollama; `allow_skip: true`.

DAG Pipeline Declared by `pipeline.yaml` II

9. **LLM Translations** (`scripts/pipeline/stage_06_llm_review.py --translations-only`; tags `llm`, same dependency edges) — multilingual abstract expansion.
10. **Copy Outputs** (`scripts/pipeline/stage_05_copy.py`) — reproducible snapshots into canonical `output/<project>/`.

DAG Pipeline Declared by `pipeline.yaml` I

Two LLM nodes intentionally share one script module with orthogonal CLI switches; both depend only on validation so they can parallelize logically while remaining optional.

Executive reporting (`scripts/pipeline/stage_07_executive_report.py`) is **not** a YAML node inside the single-project executor. `--all-projects` / `execute_multi_project.py` invokes it once after iterating projects, consolidating cross-project KPIs dashboards.

DAG Pipeline Declared by `pipeline.yaml` I

Topological order therefore differs slightly from declaration order (e.g., copy executes after validation even though stage `stage_05_copy.py` precedes `stage_06_llm_review.py` lexically).

Infrastructure vs project tests. Splitting pytest invocations isolates flaky infra regressions (`MAX_TEST_FAILURES` knobs) from zero-tolerance gates on domain code (`max_project_test_failures` default 0 declared in YAML front-matter/testing blocks).

Stage 02 illustration. The analysis stage is deliberately concrete rather than a hypothetical diagram factory: each canonical project ships real behaviour at this node. `template_autoresearch_project` runs readiness validation; `template_search_project` merges remote literature JSON, generates scripted figures (`y_generate_search_figures.py`), and writes manifests; `template_code_project` emits optimization plots; and `template_prose_project` triggers structural validation scaffolding. The pipeline shape is identical across all four—only the Stage 02 payload differs—which is exactly

what lets one orchestrator serve heterogeneous research domains. The live public roster is generated in `active_projects.md`.

run.sh |

Thin wrapper invoking `python -m infrastructure.orchestration`. Offers:

run.sh |

- per-project staged execution,
- chained digits (234 shorthand),
- multi-project grid (a–d presets),
- graceful quit / resume parity with `scripts/README.md`.

Selecting `d` **alone** after a passing multi-project run exits immediately once summaries print—avoiding repetitive menu redraw.

`secure_run.sh`

Executes Python `secure` path: standard pipeline artefact reproduction **then** invokes `run_secure_pipeline` for steganographic PDF hardening (`infrastructure.steganography`). Original PDFs stay immutable; hardened companions carry QR overlays plus hash manifests sidecars.